Presenting... Give control Stop presenting

Outline

- Testbench
- Sequential Logic Design
- Non-blocking assignment
- Flipflop & Latch
- Shift Registers
- Testbench for sequential logic circuits

11:55

Siemens FV Diploma: Verilo... 31:13
AF
Using Custom Setup

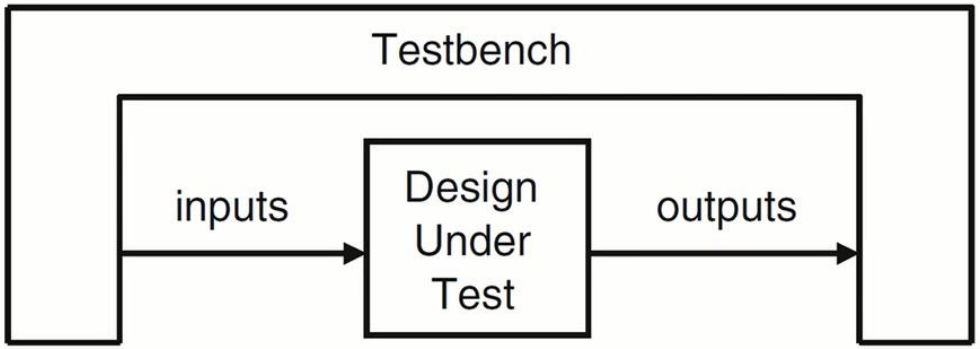
KE
Elsaid, Kareem (DI SW ICS ...

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Testbench

- Testbench is a program written for the purpose of verifying the functionality of the hardware design
- The testbench autogenerate the test patterns for the design under test (DUT) instead of forcing test vectors manually during the simulation



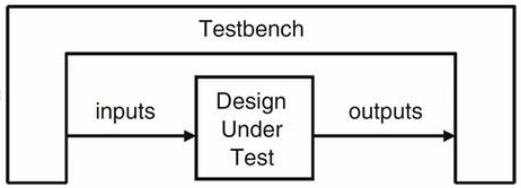
12:20

Siemens FV Diploma: Verilo... 31:37
AF
Using Custom Setup

KE
Elsaid, Kareem (DI SW ICS ...

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Half Adder Testbench




```
module half_adder_tb();
 reg [3:0] a_tb, b_tb;
 wire [3:0] y_dut;

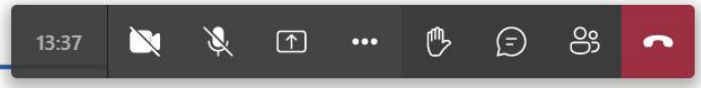
 half_adder DUT(a_tb, b_tb, y_dut);

 integer i;
 initial begin
  for (i = 0; i < 100; i = i+1) begin
    a_tb = $random;
    b_tb = $random;
    #10;
  end
  $stop;
 end

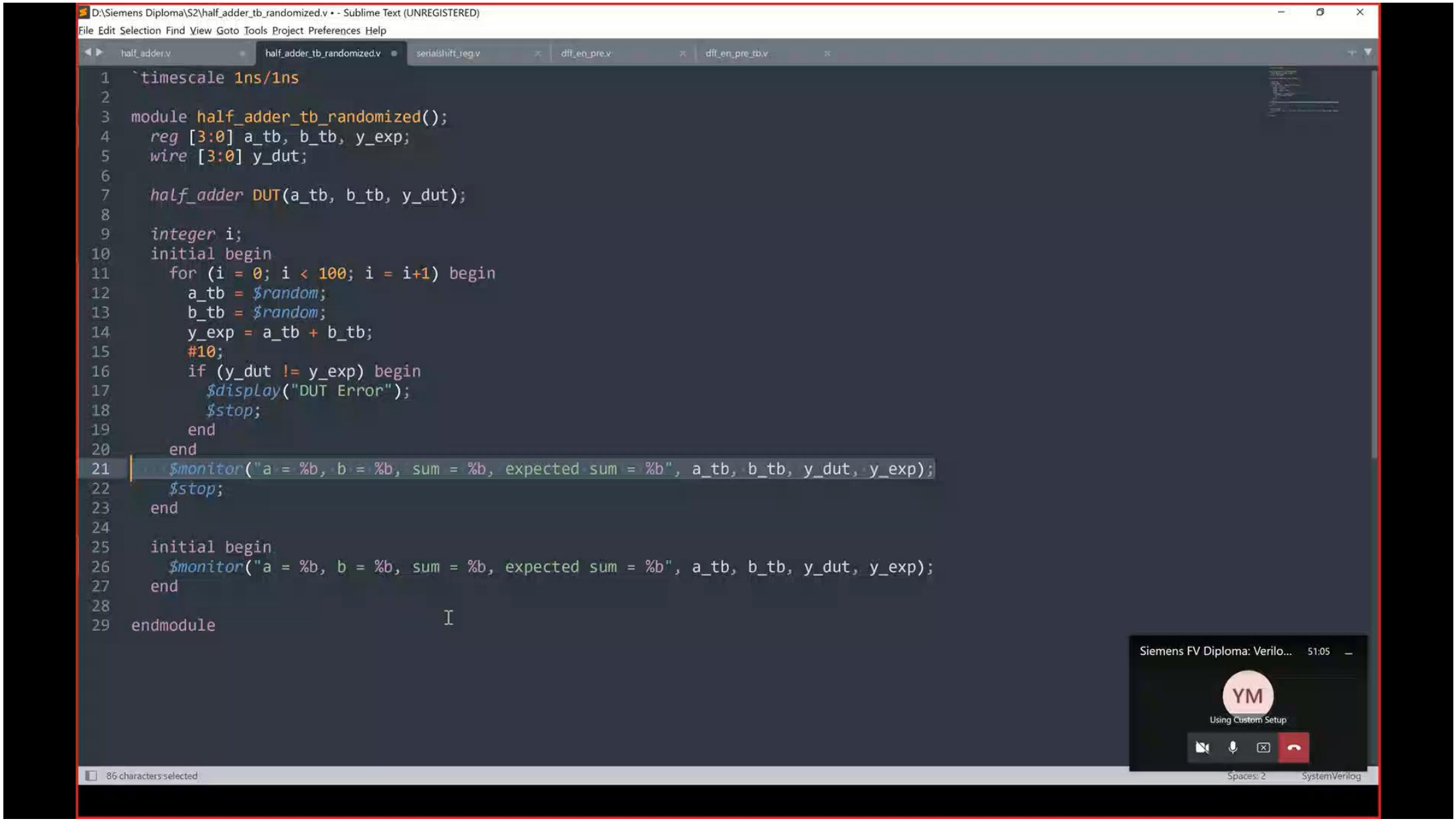
 initial begin
  $monitor("a = %h, b = %h, sum = %h", a_tb, b_tb, y_dut);
 end
endmodule
```

```
module half_adder(A, B, C);
 input [3:0] A, B;
 output [3:0] C;

 assign C = A + B;
endmodule
```



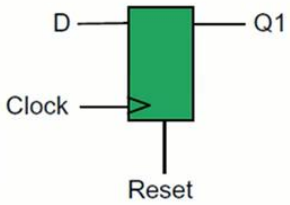


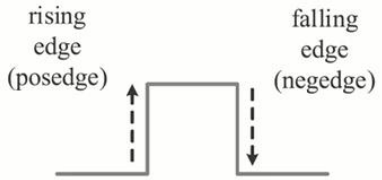

D:\Siemens Diploma\S2\half_adder_tb_randomized.v • - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
half_adder.v
half_adder_tb_randomized.v
serialshift_reg.v
dff_en_pre.v
dff_en_pre_tb.v
1 `timescale 1ns/1ns
2
3 module half_adder_tb_randomized();
4 reg [3:0] a_tb, b_tb, y_exp;
5 wire [3:0] y_dut;
6
7 half_adder DUT(a_tb, b_tb, y_dut);
8
9 integer i;
10 initial begin
11 for (i = 0; i < 100; i = i+1) begin
12 a_tb = $random;
13 b_tb = $random;
14 y_exp = a_tb + b_tb;
15 #10;
16 if (y_dut != y_exp) begin
17 $display("DUT Error");
18 $stop;
19 end
20 end
21 $monitor("a = %b, b = %b, sum = %b, expected sum = %b", a_tb, b_tb, y_dut, y_exp);
22 $stop;
23 end
24
25 initial begin
26 $monitor("a = %b, b = %b, sum = %b, expected sum = %b", a_tb, b_tb, y_dut, y_exp);
27 end
28
29 endmodule
Siemens FV Diploma: Verilo... 51:05
YM
Using Custom Setup
86 characters selected
Spaces: 2
SystemVerilog

Sequential Logic Design

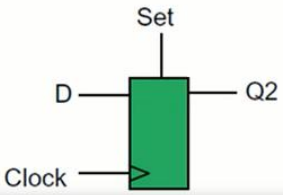
```
1// Syntax for synchronous (sequential) logics
2always @([posedge or negedge] events) begin
3  statements
4end
```

```
always @(posedge Clock or posedge Reset)
  if (Reset)
    Q1 <= 0;
  else
    Q1 <= D;
```






```
always @(posedge Clock or posedge Set)
  if (Set)
    Q2 <= 1;
  else
    Q2 <= D;
```



37:36
Siemens FV Diploma: Verilo... 56:54
Using Custom Setup
Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Non-blocking assignment

```
module comb_ct (A, B, C1, C2, C3, F);
  input A, B, C1, C2, C3;
  output reg F;

  reg z1;

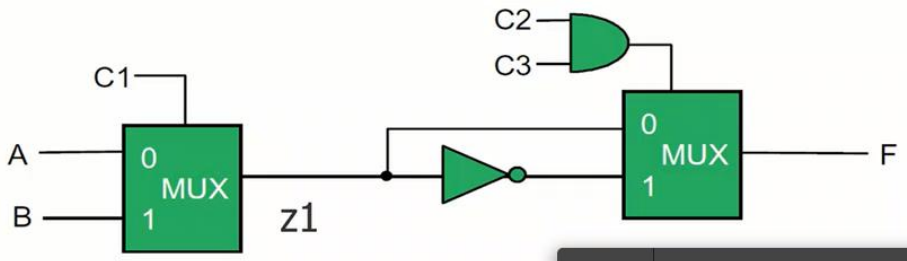
  always@(*) begin
    if (c1)
      z1 = B;
    else
      z1 = A;

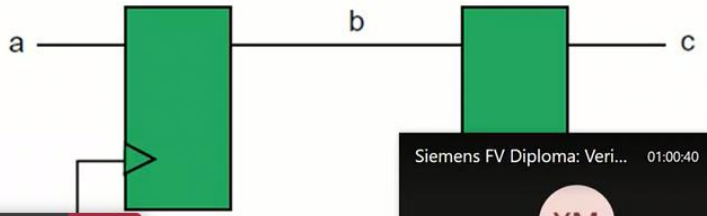
    if (c2 & c3)
      F = ~z1;
    else
      F = z1;
  end
endmodule
```

```
module seq_ct (a, Clock, c);
  input a, Clock;
  output reg c;

  reg b;

  always@(posedge Clock) begin
    b <= a;
    c <= b;
  end
endmodule
```





41:23

Siemens FV Diploma: Veri... 01:00:40

YM

Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Non-blocking assignment

```
module comb_ct (A, B, C1, C2, C3, F);
  input A, B, C1, C2, C3;
  output reg F;

  reg z1;

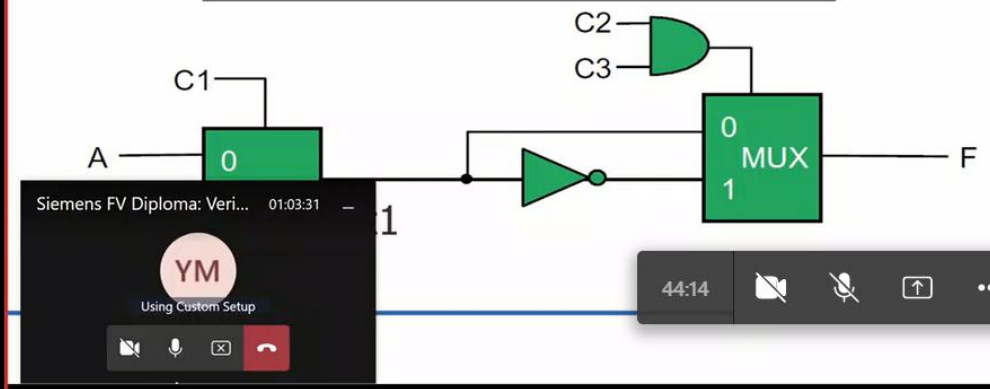
  always@(*) begin
    if (c1)
      z1 = B;
    else
      z1 = A;

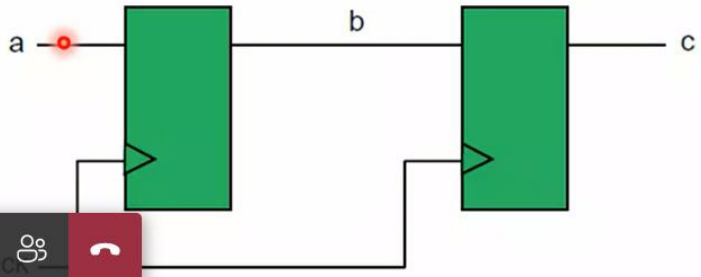
    if (c2 & c3)
      F = ~z1;
    else
      F = z1;
  end
endmodule
```

```
module seq_ct (a, Clock, c);
  input a, Clock;
  output reg c;

  reg b;

  always@(posedge Clock) begin
    b <= a;
    c <= b;
  end
endmodule
```





Siemens FV Diploma: Veri... 01:03:31
YM
Using Custom Setup
44:14

Simulation Race

```
always @(posedge clock)
  b = a;

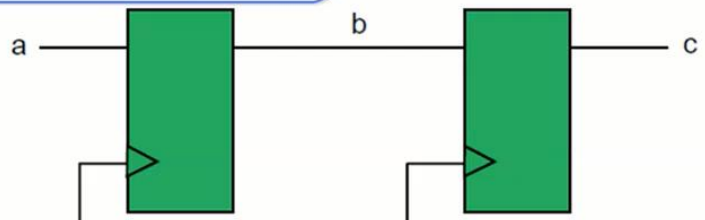
always @(posedge clock)
  c = b;
```

```
always @(posedge Clock)
  b <= a;

always @(posedge Clock)
  c <= b;
```

Simulation race fix

Non-blocking assignment



Siemens FV Diploma: Veri... 01:16:37

A1

Using Custom Setup

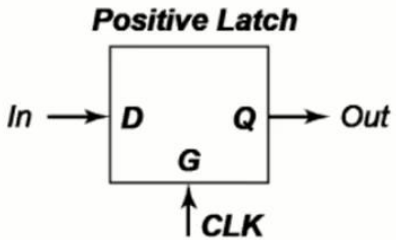
57:19

Latch

D latch | Positive Latch

```
module pos_dlatch(D, clk, Q);
 input D, clk;
 output reg Q;

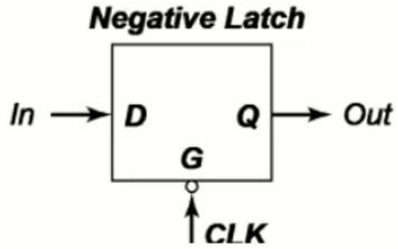
 always @(*) begin
  if (clk)
    Q <= D;
 end
endmodule
```



D latch | Negative Latch

```
module neg_dlatch(D, clkn, Q);
 input D, clkn;
 output reg Q;

 always @(*) begin
  if (~clkn)
    Q <= D;
 end
endmodule
```

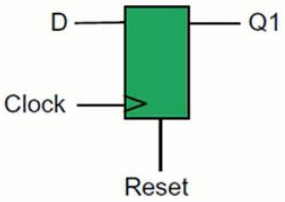


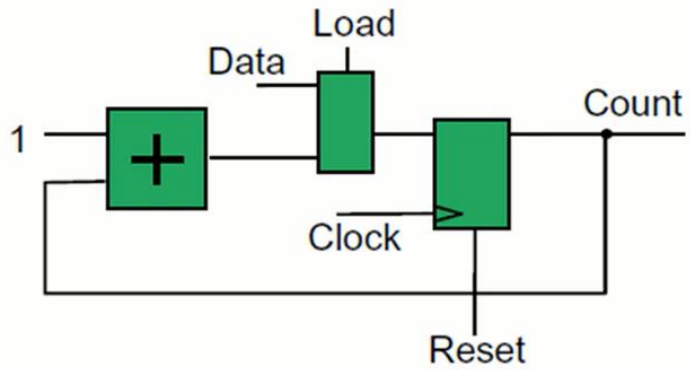
Siemens FV Diploma: Veri... 01:19:37
AM
Using Custom Setup
01:00:21
Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)
KE
Elsaid, Kareem (DI SW ICS ...

Quiz

- Parameters
  - N (Data input and output width)
  - Default: 4
- Inputs
  - Load, Clock, Reset (asynchronous)
  - Data (N bits)
- Outputs
  - Count (N bits)

```
always @(posedge Clock or posedge Reset)
  if (Reset)
    Q1 <= 0;
  else
    Q1 <= D;
```





Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

01:07:27

Siemens FV Diploma: Veri... 01:26:44
AM
Using Custom Setup

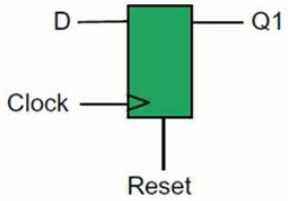
KE
Elsaid, Kareem (DI SW ICS ...

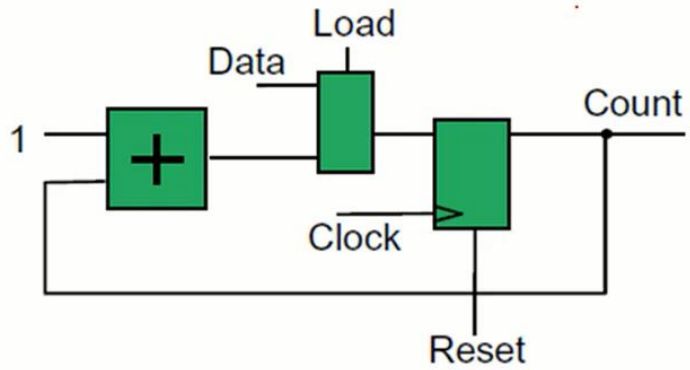
Quiz

Press Esc to exit full screen

- Parameters
  - N (Data input and output width)
  - Default: 4
- Inputs
  - Load, Clock, Reset (asynchronous)
  - Data (N bits)
- Outputs
  - Count (N bits)

```
always @(posedge Clock or posedge Reset)
  if (Reset)
    Q1 <= 0;
  else
    Q1 <= D;
```






```
module seq_ct (Load, Clock, Reset, Data, Count);
  parameter N = 4;
  input Load, Clock, Reset;
  input [N-1:0] Data;
  output reg [N-1:0] Count;

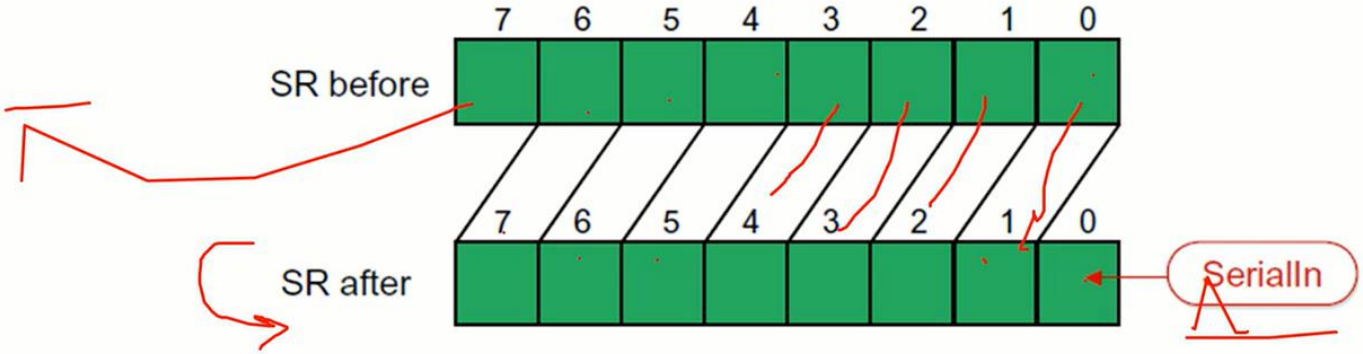
  always@(posedge Clock or posedge Reset) begin
    if (Reset)
      Count <= 0;
    else if (Load)
      Count <= Data;
    else
      Count <= Count + 1;
  end
endmodule
```

01:29:20

People are waiting in the lobby
Return to the main meeting window to see who's waiting.

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Shift registers




```
reg [7:0] SR;

always @(posedge Clock or posedge Reset)
  if (Reset)
    SR <= 8'b0;
  else
    SR <= {SR[6:0],SerialIn};
```


Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

01:37:39

Siemens FV Diploma: Veri... 01:56:56

Using Custom Setup

KE

Forever loop

- The forever statement executes an infinite loop of a statement or block of statements.
- If the loop contains only one statement, the begin ... end statements may be omitted

Syntax

```
forever
  begin
    ... statements ...
  end
```

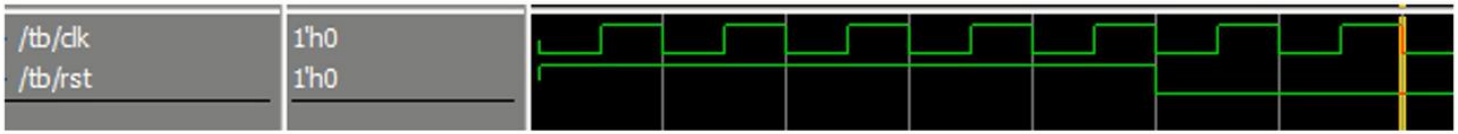

Siemens FV Diploma: Veri... 01:59:36
AM
Using Custom Setup

Clock & Reset Generation

```
module tb();
 reg clk, rst;

 initial begin
  clk = 0;
  forever
    #1 clk = ~clk;
 end

 initial begin
  rst = 1;
  #10
  rst = 0;
  #50
  /*
   .
  <other_input_test_stimulus>
   .
  */
 end
endmodule
```



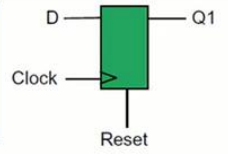
01:41:35

Siemens FV Diploma: Veri... 02:00:52
AM
Using Custom Setup

KE
Elsaid, Kareem (DI SW ICS ...

Testbench for D Flip-flop

```
always @(posedge Clock or posedge Reset)
  if (Reset)
    Q1 <= 0;
  else
    Q1 <= D;
```




```
module dff_tb();

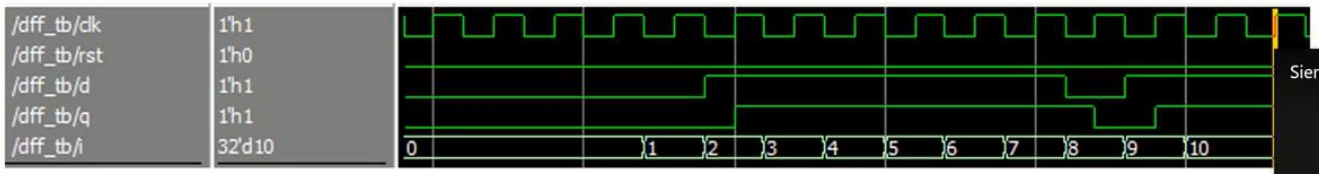
 reg clk, rst, d;
 wire q;

 integer i = 0;

 dff dut(clk, rst, d, q);

 initial begin
  clk = 0;
  forever
    #1 clk = ~clk;
 end

 initial begin
  rst = 1;
  d = 0;
  #50
  rst = 0;
  #100;
  for (i = 0; i < 100; i = i+1) begin
    d = $random;
    #2;
  end
 $stop;
 end
endmodule
```



Siemens FV Diploma: Veri... 02:05:16
AM
Using Custom Setup

Testbench Quiz

For your reference

```
module dff_tb();

 reg clk, rst, d;
 wire q;

 integer i = 0;

 dff dut(clk, rst, d, q);

 initial begin
  clk = 0;
  forever
    #1 clk = ~clk;
 end

 initial begin
  rst = 1;
  d = 0;
  #50
  rst = 0;
  #100;
  for (i = 0; i < 100; i = i+1) begin
    d = $random;
    #2;
  end
 $stop;
 end
endmodule
```

```
module dff_en_pre(E, D, Q, clk, PRE);
 input E, D, clk, PRE;
 output reg Q;

 always @(posedge clk or negedge PRE) begin
  if (~PRE)
    Q <= 1'b1;
  else if (E)
    Q <= D;
 end
endmodule
```

